实验二 软件设计与建模

外卖点餐与配送系统

需求驱动的面向对象设计与 UI 设计

项目名称	外卖点餐与配送系统
课程	软件工程
姓名	蒋至成
学号	241220006
日期	2026 年 11 月 6 日
Git 仓库	https://github.com/JZCTX/SEpa2026.git

本项目延续实验一的《外卖点餐与配送系统需求整理》，共覆盖 18 条已确认需求；
REQ-019（顾客评价）已拒绝，不在本设计范围内。

1 项目概况

1.1 项目选择与背景

本项目为**外卖点餐与配送系统**，延续实验一的访谈与需求整理成果（甲方：张宸源，需求分析工程师：蒋至成）。系统面向校园与周边社区，提供从浏览菜品、下单支付、商家接单制作到配送送达的完整闭环，并由平台统一管理店铺资质、异常订单与操作日志。

1.2 要解决的问题

· 多角色协同：顾客、商家、配送员、平台管理员围绕同一订单完成不同动作，权限与状态必须严格受控。

· 并发库存：高峰期多名顾客争抢同一菜品最后一份库存，必须避免超卖与负库存。

· 异常处置：售罄、改价、歇业、地址超出配送范围、拒单、超时未接单、配送失败都需要明确处理路径。

· 一致性：订单、支付、退款、库存、日志在操作失败或故障后仍可核对，不出现状态悬空。

· 可追溯：下单、接单、改价、退款等关键操作要留痕，并支持按条件查询。

1.3 项目目标

形成一个结构完整、可供后续实现的设计原型：模块边界清晰；订单状态机可执行且禁止非法转换；核心流程包含并发与异常分支；数据表与唯一约束可落地；界面覆盖四类角色的主要操作与典型异常态。

1.4 系统范围与明确不实现的内容

· 不接入真实支付，使用模拟支付网关完成支付与退款；

· 不接入第三方物流与地图路径规划，配送距离、时长按模拟值展示；

· 不做评价、优惠券、会员、发票与营销活动；

· 不做商家结算与提现；

· 一个订单只属于一个店铺，不做跨店合并下单与合并结算。

其中顾客评价（REQ-019）在实验一人工审查时已被判定为访谈中不存在、且甲方明确本期不做，因此本设计不再包含。

2 软件主要功能

2.1 角色与权限划分

系统包含四类角色。权限在 API 网关与业务模块两层校验：网关校验登录态与角色，业务模块再校验数据归属（例如商家只能操作本店订单）。权限矩阵如下。

功能 / 角色	顾客	商家	配送员	平台管理员
注册与登录	√	√	√	√
浏览、搜索菜品	√	—	—	—
购物车、下单、模拟支付	√	—	—	—
取消订单、申请退款	√（接单前）	—	—	√（异常裁定）
查询订单与跟踪状态	√（本人）	√（本店）	√（任务）	√（全部）
店铺入驻与营业状态	—	√	—	—
菜品、价格、库存管理	—	√	—	—
接单、拒单、出餐	—	√	—	—
分配配送员	—	√	—	—
接取、取餐、送达、上报失败	—	—	√	—
店铺资质审核	—	—	—	√
异常订单处理	—	—	上报	√（重新配送/退款）
销售统计、平台统计	—	√（本店）	—	√
关键操作日志查询	—	√（本店）	—	√（全部）
用户与违规处理	—	—	—	√

2.2 功能模块划分

编号	模块	主要功能	对应需求
M1	用户与权限	注册、登录、JWT 会话、四角色鉴权、密码失败锁定	REQ-001、002、016
M2	店铺与菜品	店铺入驻与审核状态、营业状态、菜品草稿/上架/下架、改价、库存与售罄	REQ-003、004、005、014
M3	订单与履约	购物车、下单（快照、幂等、防超卖）、模拟支付、接单/拒单/出餐、取消与退款、超时未接单	REQ-006、007、008、010
M4	配送	配送单创建、分配、取餐、送达、配送失败上报	REQ-009、010
M5	查询与统计	菜品/订单多条件查询、日销量与营业额、平台订单量对比	REQ-005、011、012
M6	日志与审计	关键操作记录、按条件查询、保留 180 天	REQ-013
M7	通知	订单状态通知、异常告警（站内消息模拟）	REQ-008、009
SCH	定时任务	每分钟扫描超时未接单订单并触发取消退款	REQ-008、010

2.3 需求到设计的覆盖

需求	实现模块	设计体现（图 / 界面）
REQ-001、002	M1	类图 User 继承；用例图注册登录；界面 01
REQ-003	M2、M6	类图 Shop 状态枚举；界面 08 审核、界面 06 营业开关
REQ-004	M2	类图 Dish（price/stock/status/version）；界面 07
REQ-005	M2、M5	用例图浏览与搜索；界面 02
REQ-006	M3	时序图一、活动图；界面 03
REQ-007	M3	状态图（10 状态、允许/禁止转换）
REQ-008	M3、SCH	时序图二超时分支；界面 06 接单倒计时
REQ-009	M4	类图 DeliveryTask；时序图二；界面 05
REQ-010	M3、M4	状态图退款路径；界面 04 退款入口
REQ-011	M5	查询与统计模块；界面 04、06
REQ-012	M5	类图 DailyReport；界面 06 热销/界面 08 平台统计
REQ-013	M6	类图 OperationLog；界面 08 日志表
REQ-014	M2、M3	活动图扣库存失败分支；界面 02、03、07 售罄态
REQ-015	架构	Redis 缓存、MySQL 索引、分页查询
REQ-016	M1、网关	组件图网关鉴权；登录页锁定提示
REQ-017	M3、M6	事务边界、幂等键、日志补偿
REQ-018	UI	界面 02 到 03 三步下单；状态色与提示

3 软件架构设计

3.1 总体结构

系统采用“四端接入—API 网关—业务模块—数据访问—基础设施”的分层模块化架构。表现层按角色分端：顾客 App、商家 Web、配送员 App、管理后台 Web，所有请求统一经 API 网关完成登录校验、角色鉴权、限流与路由。业务模块严格对应实验一的功能要求：用户与权限（M1）、店铺与菜品（M2）、订单与履约（M3）、配送（M4）、查询与统计（M5）、日志与审计（M6）、通知（M7），另有定时任务负责超时扫描。

模块之间只通过明确定义的接口交互，不直接读写对方的数据表。数据访问集中在 DAO 组件，底层使用 MySQL 8 保存主数据，Redis 7 保存会话、幂等键与热点库存缓存，对象存储保存店铺资质图片，模拟支付网关与消息推送作为外部系统。

3.2 UML 组件图

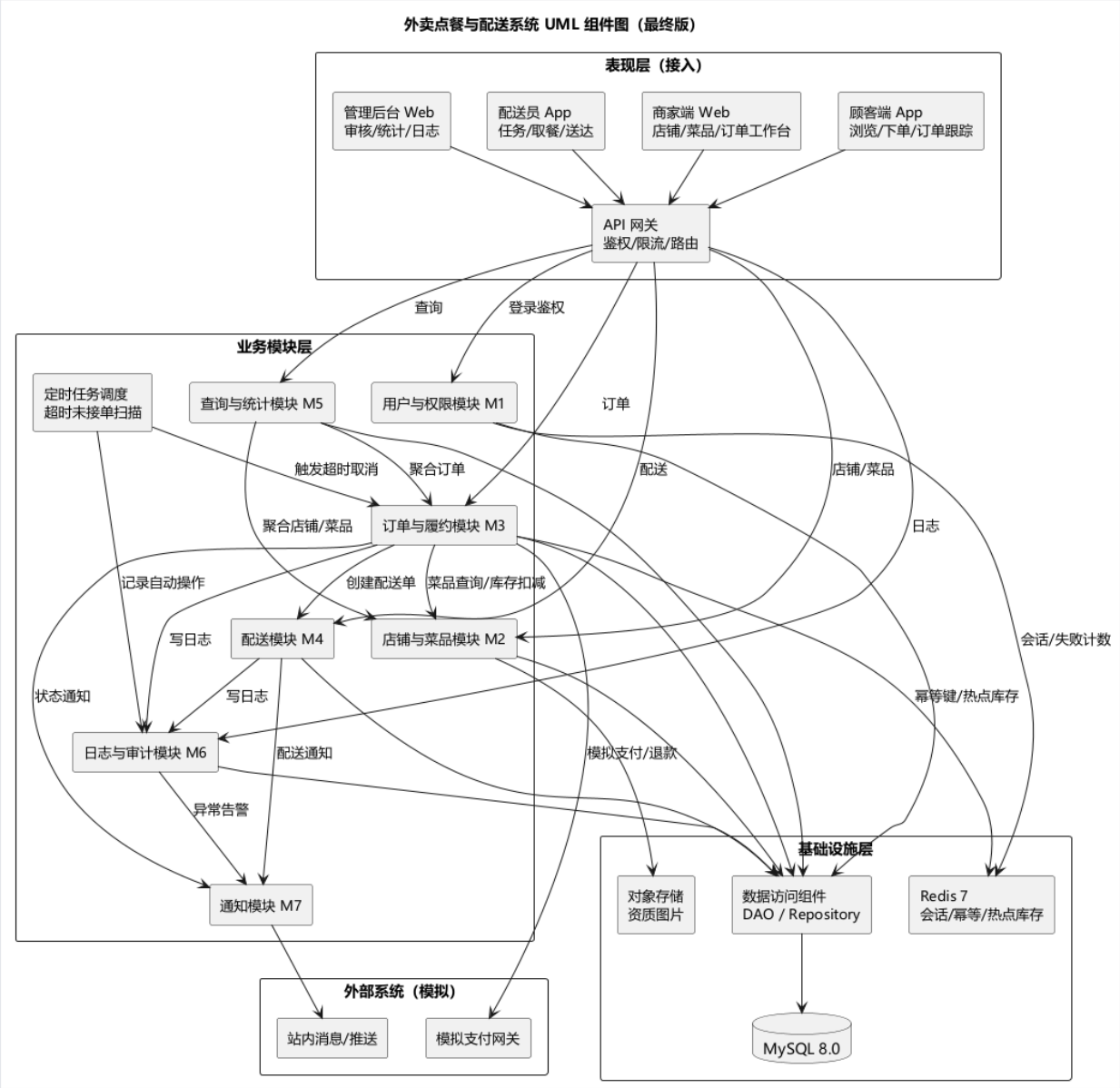


图 3-1 UML 组件图：模块划分与依赖关系

组件图表达了三个关键约束：第一，只有 API 网关对外暴露接口，四个端不直接访问业务模块；第二，订单模块依赖菜品模块完成库存校验与扣减，依赖配送模块创建配送单，依赖日志与通知完成留痕和告知；第三，所有模块经统一的数据访问组件读写 MySQL，避免各自持有连接与 SQL。

3.3 模块交互

调用方 → 被调方	方式	交互内容
顾客端 → 网关 → M3	同步 REST	提交订单、支付、取消、查询本人订单
M3 → M2	同步内部接口	校验店铺营业与菜品在售；原子扣减/回补库存
M3 → M4	同步	支付成功后创建配送单；商家分配骑手

调用方 → 被调方	方式	交互内容
M3 → 模拟支付网关	同步	支付与退款，返回交易号与结果
M3 → M6	异步	下单、接单、取消、退款等状态变更留痕
M3、M4 → M7	异步	向顾客/商家/骑手推送状态通知
SCH → M3	定时（每分钟）	扫描已支付超 15 分钟未接单订单并触发退款
M5 → M3、M2	只读	聚合订单与菜品数据生成统计，不修改主数据
网关 → M1	同步	登录、令牌签发与角色解析

3.4 关键设计决策与理由

（1）订单模块是唯一的状态权威。订单状态只允许由订单服务按其状态机变更，商家端、配送端、管理端都只能调用订单服务接口，不能直接改库。这样可避免同一订单在不同端出现状态分叉，也便于集中写日志。

（2）库存能力内聚在菜品模块。菜品模块对外只暴露“校验并扣减”“回补”两个原子操作，内部使用版本号乐观锁与条件更新（库存大于等于购买数量才扣减），保证并发下不超卖；Redis 只承担幂等键与热点缓存，库存真值始终以 MySQL 为准，避免缓存与数据库不一致。

（3）幂等键前置，唯一约束兜底。创建订单先查询请求携带的幂等键：已存在则直接返回原订单，不重复扣库存；数据库再以订单号、幂等键唯一约束兜底，防止并发下重复写入。这直接对应 REQ-006 的“重复提交返回原结果”。

（4）快照与实时数据分离。订单明细保存下单时的菜品名称与单价快照，商家后续改价、下架只影响新订单，历史订单金额保持不变（REQ-004）。

（5）支撑模块异步化。日志与通知以异步方式处理：日志写入失败只告警、不回滚交易，保证主流程可用（REQ-013）；通知失败可重试，不阻塞订单状态流转。

（6）数据访问集中。DAO 统一收敛 SQL 与事务边界，后续可替换存储或增加读写分离，业务模块不感知具体存储实现。

3.5 数据存储与一致性设计

系统以 MySQL 关系模型保存主数据，关键表与约束如下。

表	关键字段	约束与说明
user	id、phone、password_hash、role、fail_count、lock_until	phone 唯一；密码不可逆哈希；失败计数与锁定时间用于 REQ-016
shop	id、merchant_id、name、license_no、status、audit_remark	一个商家对应一个店铺；status 为待审核/营业/歇业/驳回
dish	id、shop_id、name、category、price、stock、status、version	(shop_id, name) 唯一；version 乐观锁；status 含草稿/在售/下架/售罄

表	关键字段	约束与说明
cart_item	customer_id、dish_id、quantity	(customer_id, dish_id) 唯一，重复加购累加数量
orders	order_no、customer_id、shop_id、status、total_amount、address_snapshot、idempotency_key	order_no、idempotency_key 唯一；保存地址快照
order_item	order_id、dish_id、dish_name_snapshot、unit_price_snapshot、quantity、subtotal	下单快照，不随菜品改价变化
payment	order_id、trade_no、amount、status、paid_at	order_id 唯一，一单一条支付记录
refund	order_id、amount、reason、status、refunded_at	累计退款不超过实付金额
delivery_task	order_no、courier_id、status、fail_reason	同一订单仅一条有效配送单（唯一约束）
operation_log	operator_id、role、action、target_type、target_id、before_state、after_state	保留 180 天，支持按人/时间/对象查询
notification	user_id、title、content、read_flag、created_at	站内消息，未读标记
daily_report	shop_id、report_date、order_count、gmv	(shop_id, report_date) 唯一，按日汇总

一致性规则：创建订单、扣减库存、写幂等键在同一事务边界内完成，任一失败整体回滚；支付成功后订单状态与支付单同事务写入；退款时订单状态、退款记录、库存回补同事务提交；统计按“已完成”订单口径计算，允许分钟级延迟；日志异步补写，失败告警但不影响业务结果。

4 UML 设计图

按实验要求，本设计同时给出结构性图（类图、组件图）、行为性图（用例图、状态图、活动图）与交互性图（时序图），共 7 张，源文件与 PNG 均保存在仓库 UML 目录。其中类图为必画图，另外两类各选一种以上。

4.0 图的分类、选型与阅读方式

本项目的特点是“多角色协同 + 强状态约束 + 并发库存”，因此选型标准只有一条：这张图能否直接表达一条需要人工确认的设计约束。三类图的取舍如下。

· 结构性图：类图（必画）表达领域实体与关键字段，组件图表达模块边界与依赖。对象图是类图在某一时刻的实例快照，可由类图与状态图推导，不再单画；包图的内容已由类图的 7 个包体现；部署图与复合结构图面向硬件拓扑与内部结构，本项目为单体部署，信息量很低。

· 行为性图：用例图定角色与功能边界，状态图定订单 10 个状态与禁止转换，活动图定下单校验的先后顺序。三者分别回答“谁能做什么”“什么状态能变成什么”“按什么顺序校验”。

· 交互性图：选时序图表达跨模块调用的先后关系与异常分支。通信图与时序图语义相近但更偏对象结构，本项目更关心调用顺序，故只画时序图；交互概览图可由活动图与时序图组合得到，留作后续补充。

阅读约定：箭头 A → B 表示 A 调用或依赖 B；状态图箭头文字为“事件 [守卫条件] / 动作”；时序图实线为同步调用、虚线为返回，alt 框内为异常分支。

类别	图号与图名	在本次设计中的用途
结构性图	图 4-1 UML 类图	领域实体、属性与关联；承载名称/单价快照、幂等键、库存版本号等关键字段
结构性图	图 3-1 UML 组件图	模块划分与依赖：四端 → API 网关 → 七个业务模块 → DAO → 存储
行为性图	图 4-2 UML 用例图	四类参与者与 16 个用例，标注 2 个 include 公共校验用例
行为性图	图 4-3 UML 状态图	订单 10 个状态、14 条允许转换与 4 类禁止转换
行为性图	图 4-6 UML 活动图	下单校验顺序与库存扣减失败的分支返回
交互性图	图 4-4 UML 时序图一	下单与支付的正常路径，以及幂等、歇业、库存不足三条异常路径
交互性图	图 4-5 UML 时序图二	接单到送达，以及超时未接单、骑手上报失败两条异常路径

4.1 UML 类图（结构性图）

类图按 7 个业务包组织，共 17 个类、5 个状态枚举。User 为抽象基类，派生顾客、商家、配送员、管理员；Order 聚合订单明细，并保存地址快照与幂等键；OrderItem 保存菜品名称与单价快照；Dish 带 version 字段用于并发扣减；DeliveryTask 与 Order 一对一；OperationLog 与 Notification 关联到用户。每个状态字段都对应枚举，避免散落的字符串常量。

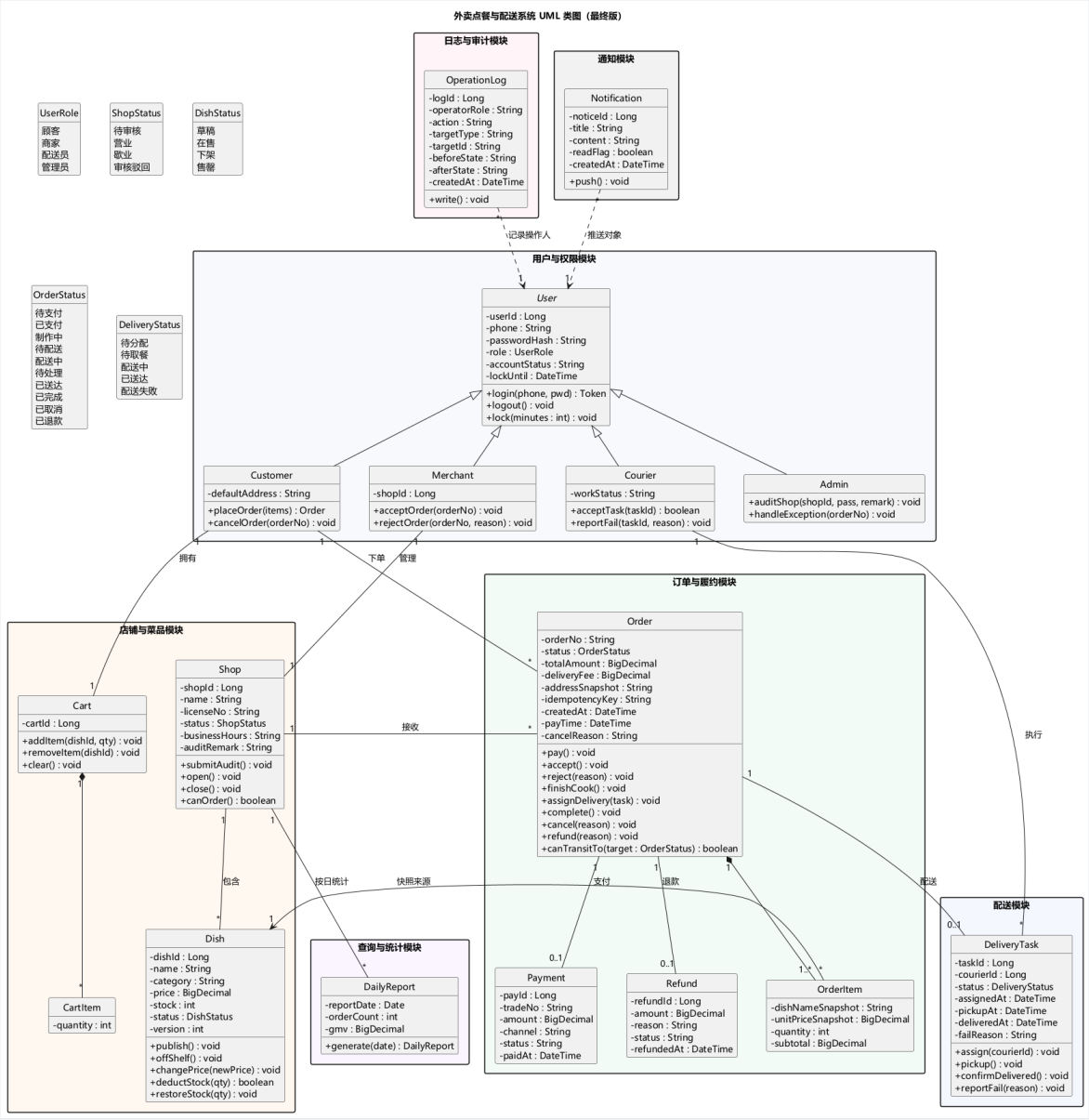


图 4-1 UML 类图

4.2 UML 用例图（行为性图）

用例图包含 4 类业务参与者与 16 个用例。顾客侧覆盖浏览、下单、支付、退款与查询；商家侧覆盖店铺、菜品、接单出餐、分配配送与统计；配送员侧覆盖配送执行与失败上报；管理员侧覆盖审核、异常裁定、统计与日志。下单与退款都包含“校验库存与幂等”这一公共用例，体现异常校验不是可选步骤。

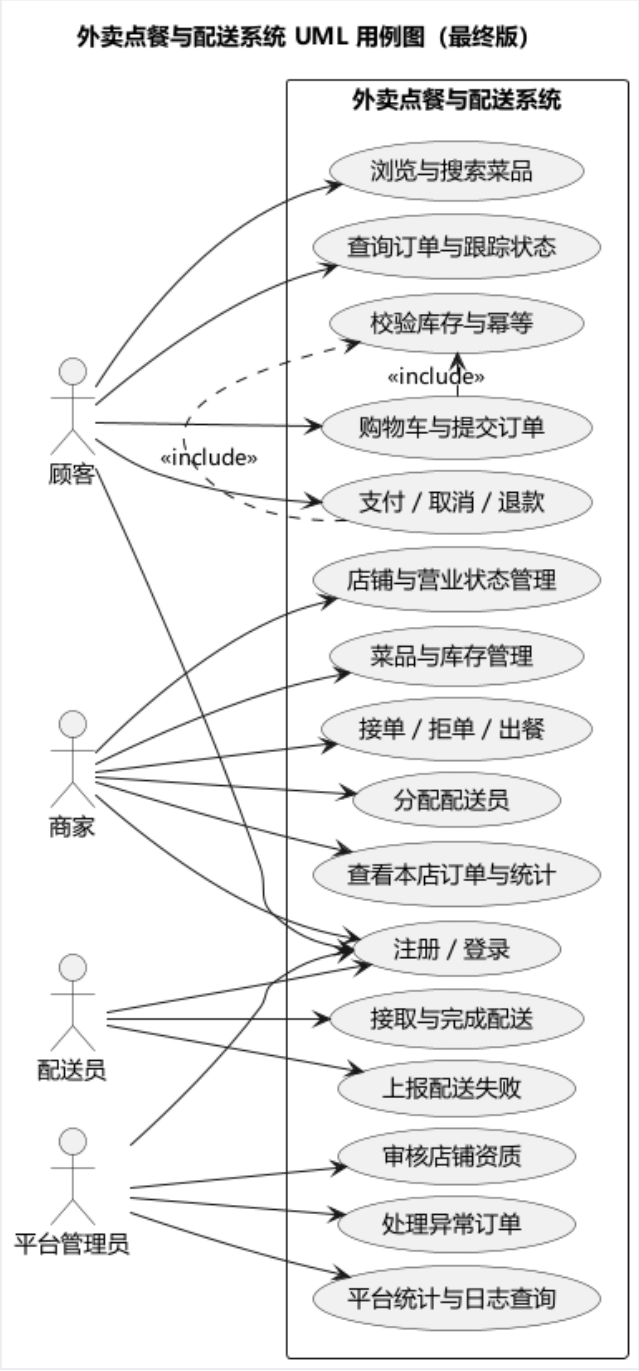


图 4-2 UML 用例图

4.3 UML 状态图（行为性图）

订单共 10 个状态，其中“待处理”是配送失败后的过渡状态，必须由管理员在限定时间内选择重新配送或退款，不能直接完成。状态图的箭头标注了触发事件与守卫条件，例如“商家接单”要求商家有权限，“重新配送”要求骑手已重新分配。

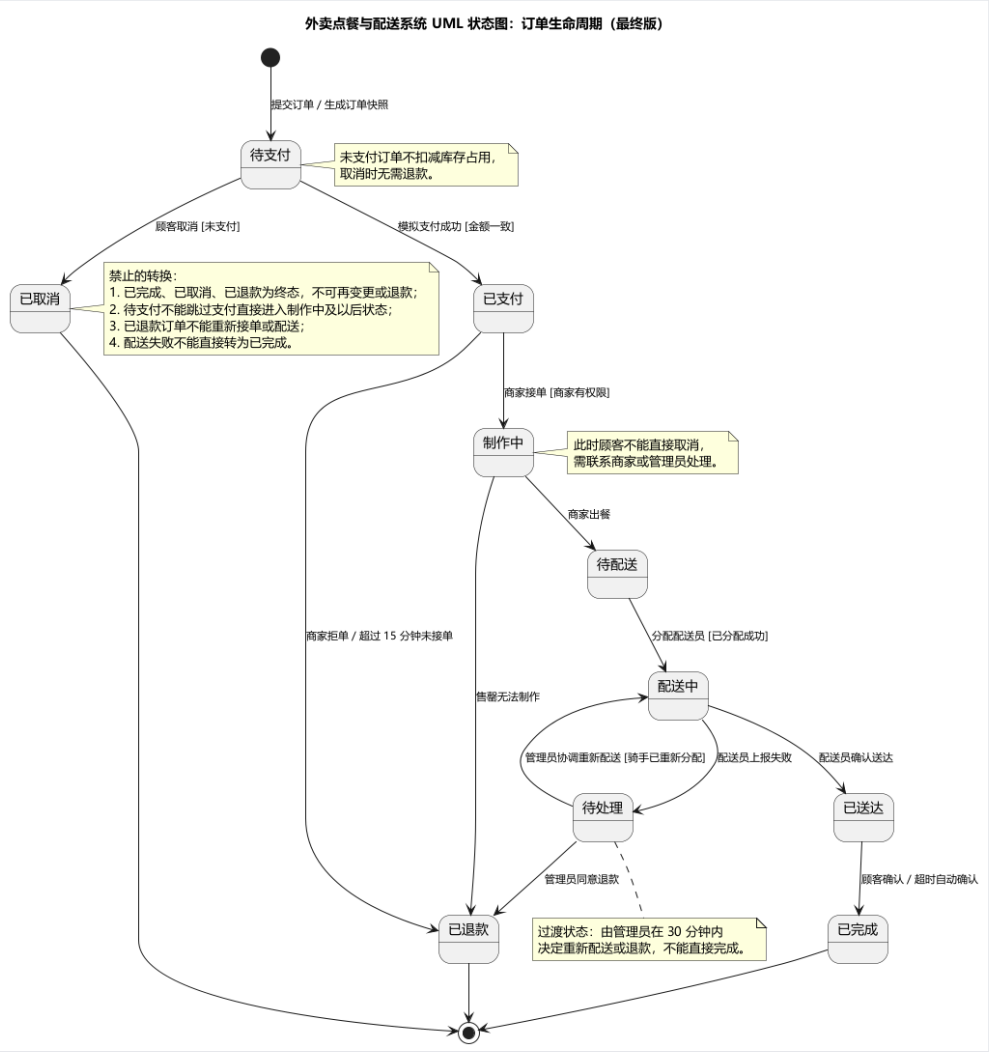


图 4-3 UML 状态图

状态转换的触发条件与禁止情况汇总如下表，可直接作为后续实现阶段的验收清单。

当前状态	允许转换	触发 / 守卫条件	禁止示例
待支付	已支付、已取消	支付成功且金额一致；顾客主动取消	不能跳过支付进入制作中
已支付	制作中、已退款	商家接单；商家拒单或超 15 分钟未接单	不能直接进入配送中
制作中	待配送、已退款	商家出餐；售罄无法制作	顾客不能直接取消
待配送	配送中	已成功分配配送员	不能直接置为已完成
配送中	已送达、待处理	骑手确认送达；骑手上报失败	配送失败不能直接完成
待处理	配送中、已退款	管理员重新分配骑手；管理员同意退款	不能直接标记已完成
已送达	已完成	顾客确认或超时自动确认	已送达不能再次退款
已取消、已退款、已完成	无	终态	任何再次变更均拒绝

状态机的落地方式。数据库只保存状态枚举值，不保存“当前阶段说明”之类的冗余文本；服务层在每次变更前调用订单实体的 canTransitTo(目标状态)，不允许通过的转换直接返回错误码 ORDER_STATE_CONFLICT，并附带当前状态与目标状态，便于前端提示。状态变更与对应的业务动作（扣库存、写支付单、创建配送单、写日志）在同一事务内完成。

为什么把禁止转换写进设计而不是只写在文档里。如果只在需求文档里写“接单后顾客不能直接取消”，实现时很容易被各个端的快捷操作绕过；把规则放进实体方法后，四个端都只能调用同一个变更接口，异常分支自然收敛。定时任务与管理员重派走的就是同一个接口，因此不需要为后台任务单独实现一套状态逻辑。

4.4 UML 时序图一：下单与支付（交互性图）

时序图一把“正常路径”和三类异常路径画在同一张图里：幂等键已存在时直接返回原订单；店铺歇业或菜品下架时终止并提示刷新；库存扣减失败（多人抢最后一份）时回滚并提示售罄。只有扣减成功才写入订单与快照，随后进入模拟支付，支付成功后更新状态并写日志。

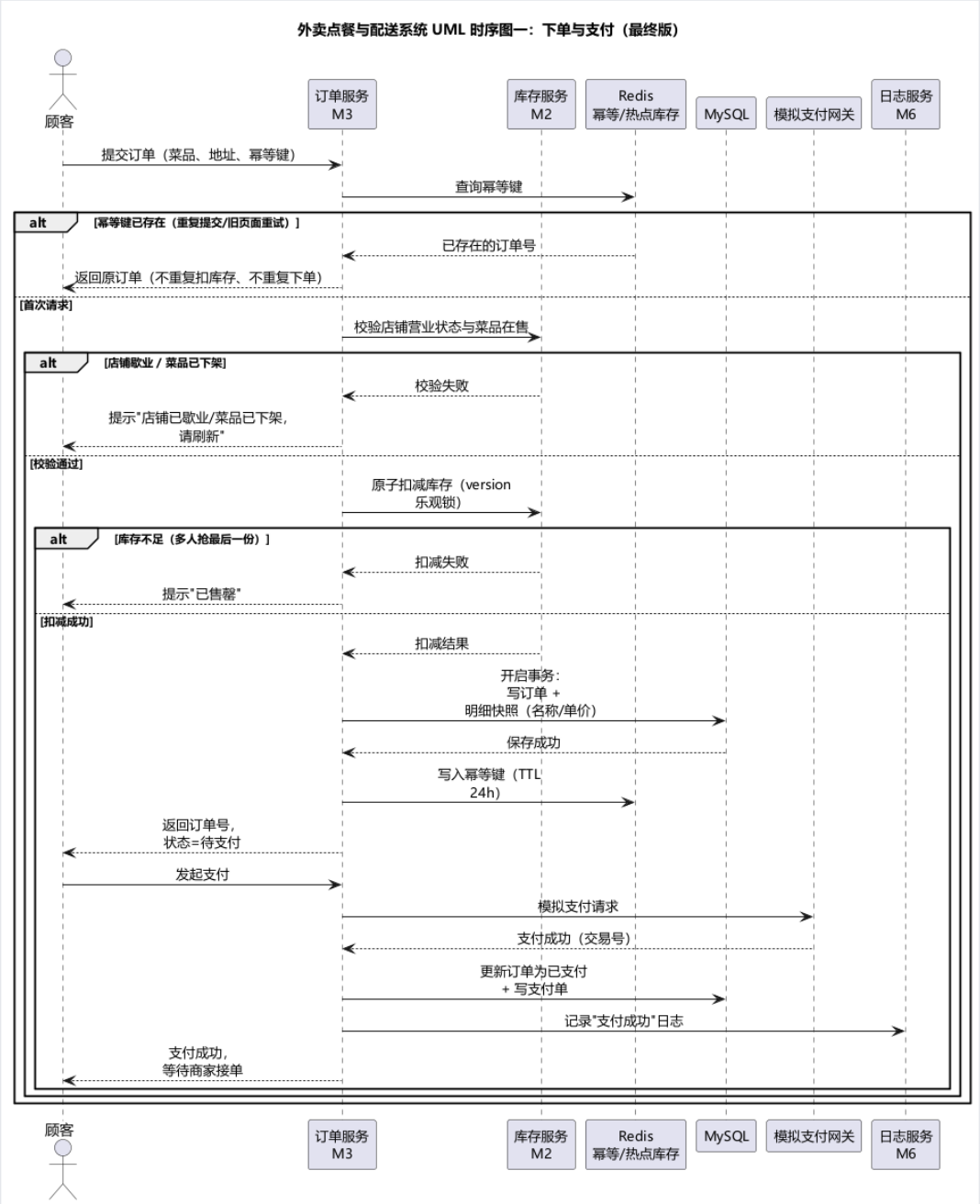


图 4-4 UML 时序图一：下单与支付

4.5 UML 时序图二：接单、配送与异常

时序图二覆盖商家接单到送达的正常链路，同时给出两条异常链路：定时任务扫描到“已支付超 15 分钟未接单”时自动退款并回补库存；骑手上报配送失败后订单进入待处理，由管理员选择重新配送或退款。这也解释了为什么状态机需要“待处理”这一过渡状态。

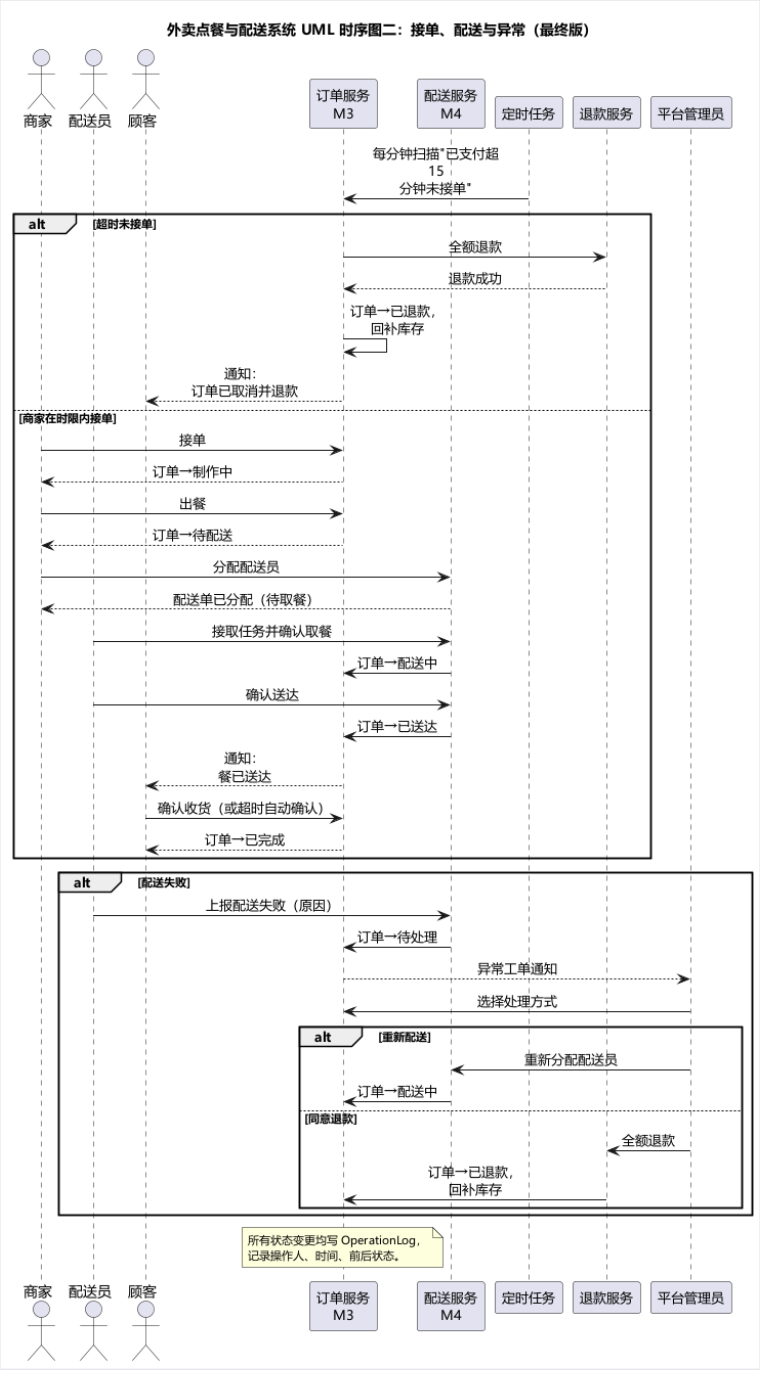


图 4-5 UML 时序图二：接单、配送与异常

4.6 UML 活动图（行为性图）

活动图描述下单校验的完整顺序：登录与角色校验、幂等校验、店铺与菜品状态校验、价格快照、库存扣减、订单落库、支付。每个判断的失败分支都有明确的返回，不存在“校验失败仍继续执行”的路径。

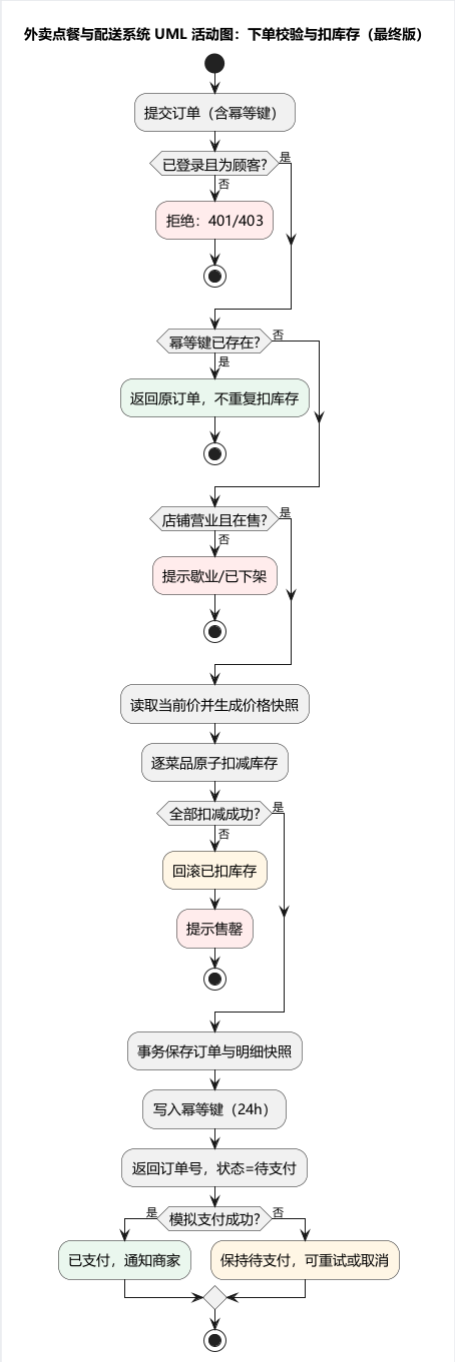


图 4-6 UML 活动图：下单校验与扣库存

5 UI 界面设计

5.1 设计原则

· 按角色分端：顾客与配送员使用移动端（390×844），商家与管理员使用桌面端（1440×900），符合实际使用场景。

· 状态可见：订单状态、菜品状态、店铺营业状态都使用统一的颜色标签（绿=正常/成功，黄=提示/待处理，红=异常/超时，蓝=中性状态）。

· 异常前置：售罄、改价、接单倒计时、配送失败等异常直接显示在界面上，不藏在二级页面。

· 统一视觉：品牌色为橙色 #FF6B35，卡片化布局、圆角与轻阴影，关键操作按钮固定在拇指可达区域。

· 可核对：金额、价格快照、操作日志在界面上可查，减少顾客与商家的争议。

5.2 顾客端：登录注册



界面 01 登录与注册

账号类型选择（顾客/商家/配送员/管理员），手机号与密码登录；商家注册需补充店铺名称与资质，审核通过后营业。页面提示“连续输错 5 次锁定 10 分钟”，对应 REQ-016 的安全要求。

5.3 顾客端：菜品浏览与搜索



界面 02 菜品浏览与搜索

顶部搜索与分类筛选支持关键词、分类组合查询（REQ-005）。店铺卡片显示营业状态；库存为 0 的菜品灰化并标注“已售罄”，加号禁用，顾客无法加入购物车（REQ-014）。歇业店铺标记“已歇业”，不能下单。

5.4 顾客端：购物车与确认订单



界面 03 确认订单

展示收货地址与配送范围校验结果、菜品明细、运费与优惠。商家改价时，页面出现黄色提示“价格由 ￥20 调整为 ￥22”；售罄菜品被移出并灰化提示。提交时再次校验库存与价格，重复点击不会生成两笔订单（REQ-006、REQ-014）。

5.5 顾客端：订单跟踪



界面 04 订单跟踪

以时间轴展示已支付、制作中、待配送、配送中、已送达五个节点，当前节点高亮；展示骑手信息与预计送达时间。商家接单后“直接取消”不可用，提示联系商家或申请退款，对应状态图的禁止转换（REQ-007、REQ-010）。

5.6 配送员端：配送任务



界面 05 配送任务

显示当前任务的取餐点、送达点、距离与预计时间，提供“确认取餐”“确认送达”与“上报配送失败”。上报失败必须选择原因，订单转入待处理，由管理员协调重派或退款（REQ-009）。

5.7 商家端：订单工作台

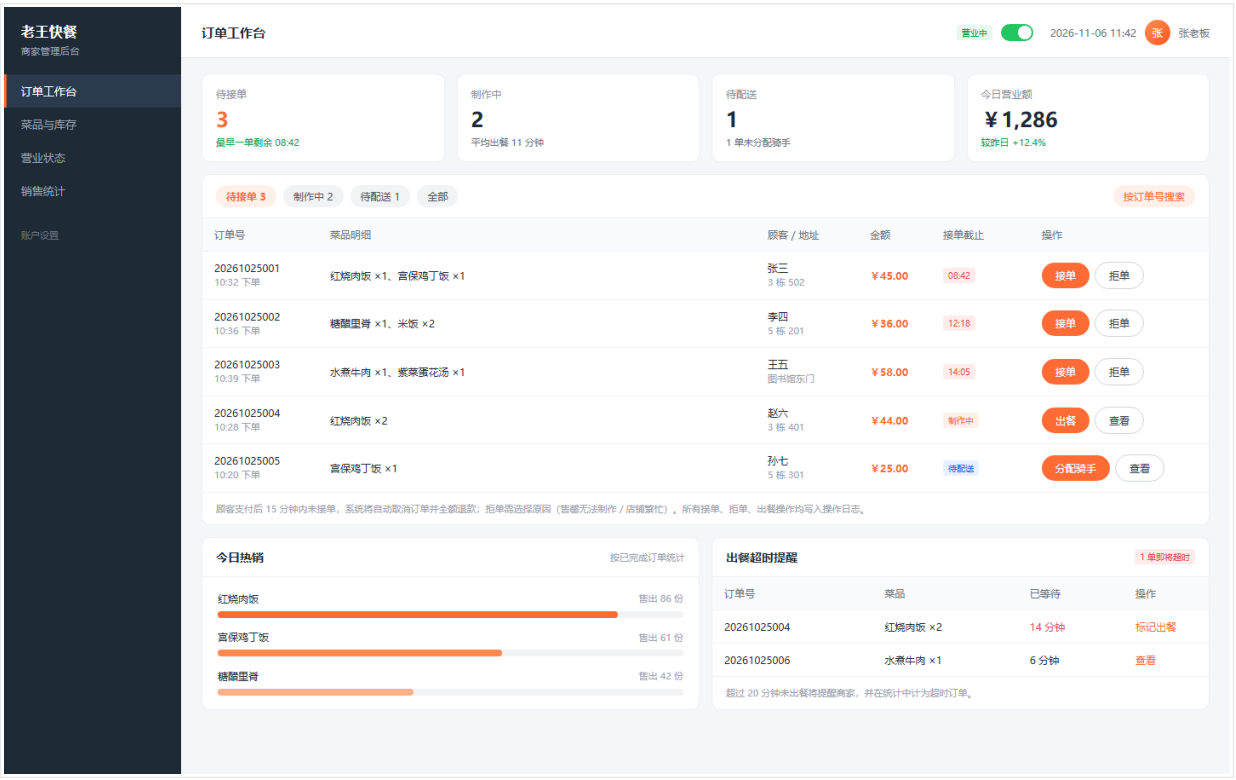


图 5-1 商家端订单工作台

工作台按“待接单、制作中、待配送”分区，待接单订单显示接单倒计时（红色），拒单需选择原因。底部提示“15 分钟未接单自动取消并退款”，并说明所有接单、拒单、出餐操作写入操作日志。右侧提供今日热销与出餐超时提醒，帮助商家安排产能（REQ-008、REQ-012、REQ-013）。

5.8 商家端：菜品与库存

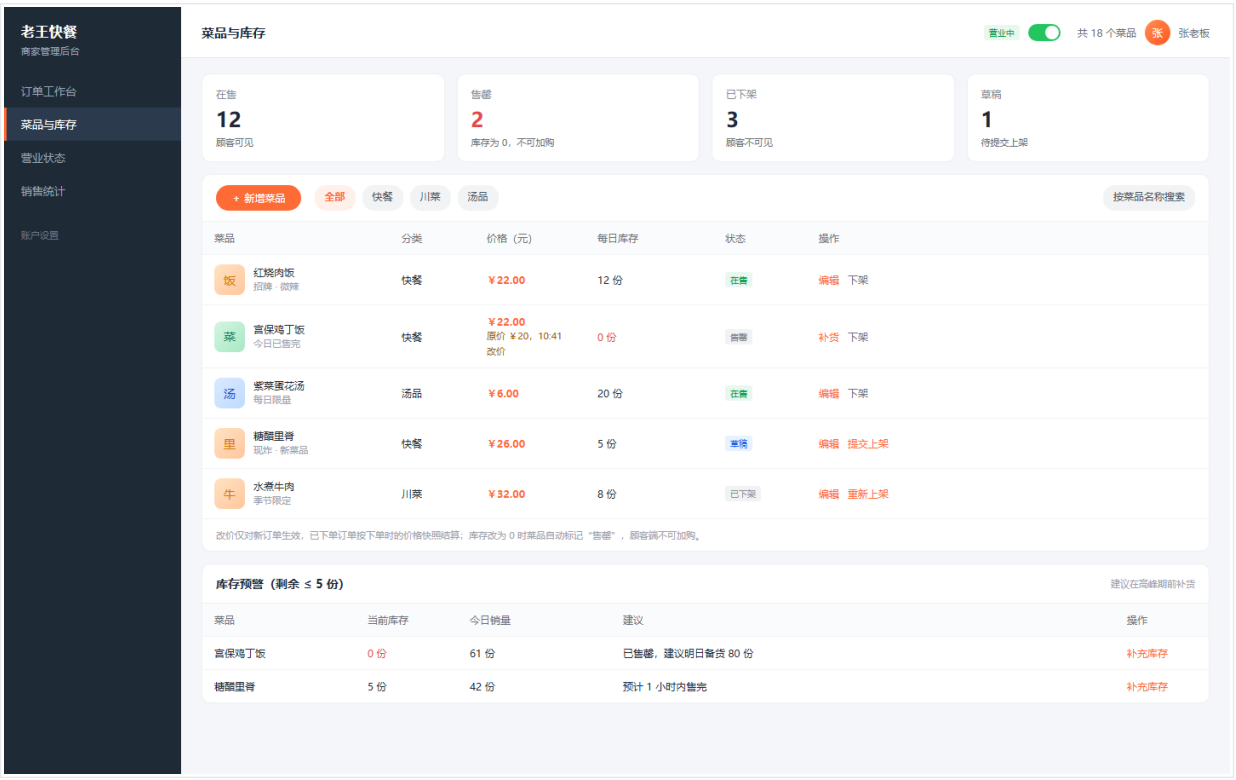


图 5-2 商家端菜品与库存

菜品表包含草稿、在售、售罄、已下架四种状态，支持新增、编辑、改价、上下架与补货。改价提示“仅对新订单生效，已下单按快照结算”；库存为 0 自动变为售罄；底部库存预警列出剩余不超过 5 份的菜品，便于提前备货（REQ-004、REQ-014）。

5.9 管理端：店铺审核与平台统计

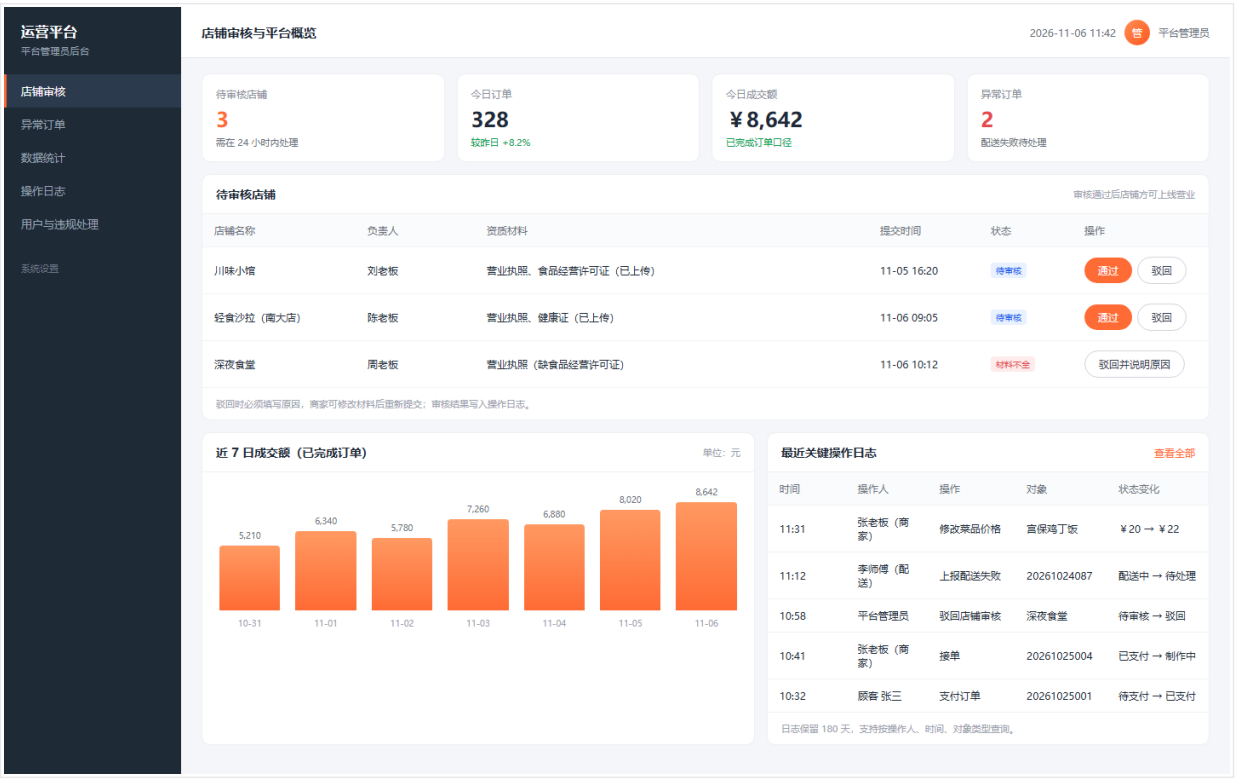


图 5-3 管理端店铺审核与统计

管理端集中处理店铺资质审核（通过/驳回并填写原因）、平台关键指标、近 7 日成交额与最近操作日志。日志表展示时间、操作人、操作、对象与状态变化，并注明保留 180 天、支持按条件查询（REQ-003、REQ-012、REQ-013）。

6 AI 辅助设计的过程与思考

6.1 使用的工具与方式

本次使用 TRAE 内置 AI 助手（DeepSeek-V4.1-Flash）辅助设计，PlantUML 1.2024.7 负责渲染，界面用 HTML/CSS 绘制后用 Edge 无头浏览器截图。使用原则是：需求事实只来自实验一的访谈记录与需求列表；AI 只做“结构转换”和“起稿”，所有状态、异常与数据约束由本人核对后决定。完整记录见仓库中的 AI使用记录.md，以下摘录关键六次交互。

6.2 逐次使用记录

编号	场景	生成结果	主要问题	人工修改后
AI-01	模块划分与用例图	4 模块、12 用例，仅正常流程	漏管理员用例与系统定时行为，未区分 include	扩为 7 模块 + 定时任务，16 用例、2 个 include
AI-02	类图	9 个类与基本关联	缺支付/退款/配送/日志，无快照、幂等键、版本号	补至 17 类 5 枚举，增加快照、幂等键、乐观锁
AI-03	订单状态机	6 状态正向链路	缺“待处理”，无禁止转换与守卫条件	10 状态、14 转换，补 4 类禁止说明
AI-04	时序图与活动图	两张正常路径图	无幂等、并发、改价、超时、配送失败分支	拆成两张时序图，补 alt 分支；活动图补失败返回
AI-05	UI 方案	通用电商布局，2 页纯文本卡片	无异常态、无角色区分、无色板	统一色板，补 8 个界面与异常态
AI-06	报告结构与术语校对	目录与术语检查	内容偏概括、缺过程细节	重排结构，回填需求编号与量化口径

AI-01 模块与用例。初稿的模块边界基本正确，但没有把“超时未接单自动处理”当成一个系统行为，也没有管理员处理异常订单的用例。我据此把定时任务列为独立组件，并把异常裁定补进管理员用例。这提醒我：AI 会把系统当成“只有用户主动操作”，而真实系统还有后台任务。

AI-02 类图。初稿只有九个类，最严重的问题是订单金额只有一个总额，无法支撑“改价不影响已下单订单”。我补充了订单明细的名称与单价快照字段、幂等键和库存版本号。这些字段不是排版问题，而是决定后续能否实现 REQ-004 与 REQ-014 的结构性问题。

AI-03 状态机。初稿把配送失败当作不存在，导致失败订单无路可走。我增加了“待处理”过渡态，并明确四条禁止转换。这里我特意没有让 AI 重画，而是自己按实验一的状态描述逐条核对，因为状态机的正确性不能靠“看起来合理”。

AI-04 时序图。初稿只有顺利路径。我要求它补充幂等、并发扣库存、超时与配送失败，但生成的分支仍不完整，最后是我按状态机逐个补的。优点是 alt 语法和参与者划分可以直接复用。

AI-05 界面。初稿是通用蓝色电商模板，没有售罄、改价、倒计时这些本项目的关键状态。我改为橙色品牌色，把异常直接画进界面；商家端增加了热销与出餐超时，菜品页增加了库存预警。AI 给的是“通用骨架”，业务特征必须自己加。

AI-06 报告。AI 的目录与术语检查有效（发现“待处理”和“配送失败”混用），但正文偏空泛。我把所有数字回查 requirements.md，并把 AI 章节改成逐条记录，避免写成泛泛的优点总结。

6.3 优缺点分析

优点：生成速度快，能在几秒内给出语法正确的 PlantUML 骨架、完整的类清单与可用的界面布局，显著减少机械劳动；对术语不一致、明显遗漏有提醒作用；修改成本低，适合快速比较多个方案。

缺点：倾向于只画正常路径，对并发、幂等、超时、状态冲突这些真正决定设计质量的部分不敏感；会补充需求中不存在的功能（如评价），必须逐条对照访谈；UML 自动布局经常连线交叉，需要人工调整；界面缺少异常态与业务特征；文字描述容易空泛。

结论：AI 适合做“第一稿”和“结构补全”，不适合做最终判断。需求取舍、状态约束、异常优先级必须由人决定。

6.4 能否设计一套“人只做检查校对”的工作流

可以接近，但不能完全做到。可落地的流程分为五步：

· 人先固定事实源：需求编号、角色权限矩阵、状态清单、异常清单，作为唯一输入；

· AI 只做格式转换：按固定模板生成 PlantUML 与界面结构，明确禁止新增需求；

· 脚本自动校验：检查每条需求是否至少被一个模块或界面覆盖、每个状态是否有入边和出边、异常清单是否都在界面出现；

· 人做判断：确认语义正确性、异常优先级与交互取舍；

· 回归沉淀：把人工修改反写进模板与提示词，形成可复用的检查清单。

其中第 2、3 步可以高度自动化，第 1、4 步仍必须由人完成，因为“要不要这个功能”“异常时优先退款还是重派”属于产品决策，不是文本生成问题。因此现实目标应是“人做决策与验收，AI 做转换与自检”，而不是人完全不参与。

7 与实验一的衔接及后续工作

本设计覆盖实验一 18 条已确认需求中的全部功能与非功能要求，并通过模块、图与界面形成可追溯关系；REQ-019 已拒绝，不进入设计。与实验一相比，本实验把“文字需求”落实为可执行的模块边界、状态约束与数据约束。

后续实验三实现时的建议顺序：先完成 M1 登录鉴权与 M2 菜品管理，再做 M3 下单支付（含幂等与库存扣减），然后接入 M4 配送，最后补 M5 统计与 M6 日志；每完成一个模块，用本报告的状态表与验收口径做一次核对。

本阶段留作 TODO：真实支付渠道、地图路径规划与预计送达时间计算、消息推送通道、压力测试脚本（对应 REQ-015）、可用率统计（REQ-017）。这些不影响的当前设计的模块边界，可在实现阶段补充。

8 版本信息与提交内容

项目	内容
Git 仓库	https://github.com/JZCTX/SEpa2026.git
lab2-ai-draft	12e39df029449d249e75c7e9769fcb5c9ede028b
lab2-final	38d6abaa38cf80f93cc0901feb8a6f071c130d19
UML 源文件	UML 目录下 7 个 .puml 及对应 PNG
UI 界面	UI界面 目录下 8 张 PNG（5 张 390×844，3 张 1440×900）
AI 记录	AI使用记录.md（六次交互的完整记录）

提交压缩包结构：实验2_241220006_蒋至成/ 包含 实验报告.pdf、UML/（7 组 png+puml）、UI界面/（8 张 png）。

（完）